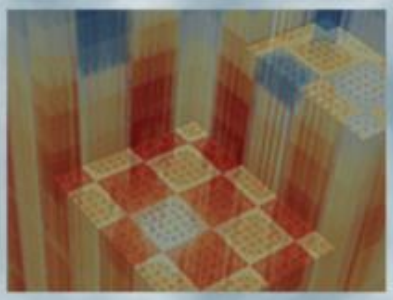
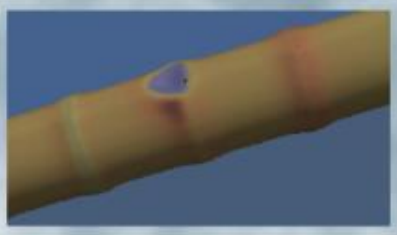
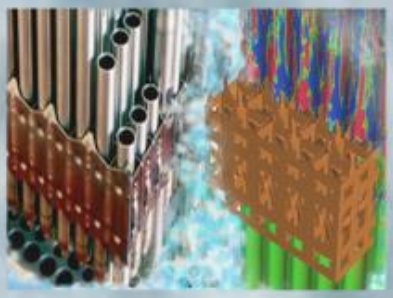
CASL-2019-1884-000











Consortium for Advanced Simulation of LWRs

VERAIn Programmer Manual

E. Walker[1]

[1]Oak Ridge National Laboratory

7/31/2019

VERA QA Document – This document was prepared under the ORNL VERA Quality Assurance Program in accordance with procedure VERA-QA-003. The OFFICIAL COPY of this document is the electronic version in the VERA Documents Repository. Before using a printed copy, verify that it is the most current version by checking the Revision ID against the electronic version.

U.S. DEPARTMENT OF ENERGY | Nuclear Energy

Revision Log

Revision	Date	Affected Pages	Revision Description
0	7/31/2019	All	Initial Release

Document pages that are:

Export Controlled:	None
IP/Proprietary/NDA Controlled:	None
Sensitive Controlled:	None
Unlimited:	All

This document was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

VERAIn Programmer Manual

Approvals:

Erik Walker — Digitally signed by Erik Walker, Date: 2019.08.12 16:49:48 -04'00'

Erik Walker, Author — **Date**: 8/12/2019

Scott Palmtag — Digitally signed by Scott Palmtag, Date: 2019.08.12 17:10:46 -04'00'

Scott Palmtag, VERAIO Product Software Manager — **Date**: 8/12/2019

Travis Lange — Digitally signed by Travis Lange, Date: 2019.08.12 17:05:31 -04'00'

Independent Reviewer — **Date**: 8/12/2019

Executive Summary

This document provides information needed by developers who will be working in the VERAIn source code. It provides a high-level overview of input design as well as guidelines for adding new inputs.

Contents

Executive Summary iv

1 VERAIn **1**

2 Style Guide **2**

2.1 Adding New Variables 2

2.2 Testing New Variables 3

3 VERARun **6**

3.1 VERARun Output 7

List of Figures

2.1 Example b10 inputs and descriptions in Directory.yml 4

2.2 Example b10 inputs in STATES.yml 5

1. VERAIn

The VERAIn repository is used to translate the user defined ASCII input file to a ParameterList XML file to be read by VERA. The ParameterList format is defined in Trilinos Teuchos package at: `https://trilinos.org/docs/dev/packages/teuchos/doc/html/index.html#TeuchosParameterList_src`. The VERAIn parser is a PERL script that translates individual variables that are specified in YAML files. These YAML files are located in the `VERAInExt/verain/scripts/Templates` directory. To perform the translation, use the parser located in the scripts directory. Run the parser with a command line like the following:

```
VERAInExt/verain/scripts/react2xml.pl $CASE.inp $CASE.xml
```

where `$CASE` is the base name of your input file. Parser error messages indicate errors in the input file. More verbose diagnostic messages are currently being added. Various input parameter examples can be found in the `VERAInExt/verain/test` directory.

2. Style Guide

2.1 Adding New Variables

This section provides instructions for adding or modifying inputs to be used in the user provided ASCII input. For most cases, to add a new variable to the input, only the template files found in `VERAInExt/verain/scripts/Templates` need to be modified. Generally, two files need to be modified for every new input variable:

- Directory.yml — gives a description of how to read the input cards
- BLOCK.yml — gives a description of how to convert input cards to parameter names for a particular BLOCK

The different BLOCK options that are available in an input file are:

`STATE, CORE, ASSEMBLY, CONTROL, INSERT, DETECTOR, EDITS, SHIFT, COBRATF, COUPLING, MPACT, BISON, MAMBA, and RUN`

For example, if a new variable is to be added to the ASSEMBLY block of the user input, it would need to be added to the ASSEMBLY section of the Directory.yml file as well as to the ASSEMBLY.yml file.

Whenever a new variable is added to the Directory.yml file, proper documentation must also be provided. The input description must precede the variable and be of the following form using LaTeX formatting:

```
#>
#>   {\bf variable\_name} detailed\_input\_name
#>   \VERAInputTable{
#>     name={detailed\_input\_name},
#>     type={variable type: Float, Boolean, Integer... etc.},
#>     need={Optional or Required},
#>     unitsdefault={Default units for this variable},
#>     unitsother={Other acceptable units for this variable},
#>     valuedefault={Default value for this variable, if there is one},
#>     valuesapplicable={Defines acceptable values, or range of values, that this variable
#>        can be defined as},
#>     limitations={A description of the limitations of this variable, if applicable},
```

```
#>     description={A detailed description of this variable and what it is used for.},
#>     notes={Any additional information about this variable that was not specified earlier}
#>   }
```

It is strongly advised that a new variable include a `_check:` section that checks to make sure that a given variable is the proper type and within the expected input variable range. An example of a two-variable input card with description is shown in Figure 2.1. This example input, `b10`, is located in STATE block. Therefore, the corresponding entries in the STATES.yml file are shown in Figure 2.2. The names given in the BLOCK.yml file will correspond to the variable name that will given in the ParameterList XML file.

2.2 Testing New Variables

Whenever a new input card is added, a test case must also be added. To add a test case, either modify an existing test case or add a new one in the `VERAInExt/verain/test` directory. It is preferable to modify an existing case, but care must be taken not to impact another test that might use the same case.

There is a special test input deck called `misc_options.inp` that contains a bunch of random input variables. The only purpose of this test is to make sure something gets written to the XML file correctly. If possible, try to use this test case instead of creating something new. Once a test case has been either modified or created, a "gold" xml file must be created to go along with it. If a new test is added, be sure to modify the `CMakeLists.txt` file in the `VERAInExt/verain/test` directory and add another entry to the long list of `ADD_REACTOR_AWARE_INPUT_PARSER_TEST` cases.

```
#>
#>    {\bf b10} b10\_fraction b10\_depletion
#>    \VERAInputTable{
#>      name={b10\_fraction},
#>      type={Float},
#>      need={Optional},
#>      unitsdefault={},
#>      unitsother={Atom fraction of B-10 in boron},
#>      valuedefault={0.199},
#>      valuesapplicable={$>=0$},
#>      limitations={},
#>      description={Boron-10 fraction in coolant},
#>      notes={}
#>    }
#>    \VERAInputTable{
#>      name={b10\_depletion},
#>      type={Boolean},
#>      need={Optional},
#>      unitsdefault={},
#>      unitsother={},
#>      valuedefault={False},
#>      valuesapplicable={True},
#>      limitations={},
#>      description={Flag to enable B-10 depletion in coolant},
#>      notes={Required when using input parameter \texttt{b10} }
#>    }
  b10:
    <<: *dtlist
    _content:
    _check:
      - arraysize()==2
      - is_float(nth(0,()))
      - is_word(nth(1,()))
      - nth(0,())>=0
```

Figure 2.1: Example b10 inputs and descriptions in Directory.yml

```
b10:
  _pltype: parameter
  _type: double
  _do:
    - copy %STATE/$(_loop)/$b10:0
  _content:

b10_depl:
  _pltype: parameter
  _type: bool
  _do:
    - copy %STATE/$(_loop)/$b10:1
  _content:
```

Figure 2.2: Example b10 inputs in STATES.yml

3. VERARun

VERARun is a set of Python scripts that allow for the easy execution of problems in VERA. The scripts are located in the VERAInExt repository in the `VERAInExt/verarun` directory.

To execute a problem using VERARun, a user simply types:

```
verarun $CASE
```

Additional command line options can be found by typing `verarun` with no arguments. Doing so will return the following options.

```
usage: verarun [-x] [-e email_addr] [-h] [-c config_file] [-N job_name] [-l]
               [-n nprocs] [-O] [--ppn cpus_per_node] [-m mem_per_process]
               [-s subdir] [-d vera_install_dir] [-v vera_version] [--verbose]
               [-W] [-w job_id] [-t walltime] [--chain] [--debug]
               [--hostname host] [-r {overwrite,readwrite}]
               [--vera-installs-dir vera_installs_dir]
               [input_path [input_path ...]]

Creates and optionally submits machine-specific VERA jobs.

positional arguments:
  input_path            path to VERA input (.inp) or XML (.xml) files

optional arguments:
  -x, --dry-run         dry run only, create but don't execute the PBS script
  -e email_addr, --email-addrs email_addr
                        comma-delimited list of email addresses to notify of
                        job completion, defaults to ${USER}@$(hostname)
  -h, --help            print detailed help
  -c config_file, --host-config-file config_file
                        override host configuration file, supercedes
                        --hostname and --vera-installs-dir
  -N job_name, --job-name job_name
                        name for the PBS job
  -l, --list-vera-versions
                        list available VERA versions
```

```
  -n nprocs, --np nprocs, --nprocs nprocs, --num-procs nprocs
                        total number of processors need for the job (mpiexec
                        -np param), defaults to value computed from input
  -O, --output-job-name
                        print the job name to stdout
  --ppn cpus_per_node, --pnode cpus_per_node
                        specify processors per node, by default this is
                        calculated
  -m mem_per_process, --pmem mem_per_process, --proc-memory mem_per_process
                        specify memory required per processor in GB, defaults
                        to undefined
  -s subdir, --subdir subdir
                        create subdir, a value of "." specifies automatically
                        generated subdir name
  -d vera_install_dir, --vera-dir vera_install_dir
                        path to VERA installation, superceding --vera-
                        installs-dir, --vera-version, and the host
                        configuration
  -v vera_version, --vera-version vera_version
                        name of VERA version to use
  --verbose             turn on verbose messaging
  -W                    wait on job last submitted via verarun, overrides -W
  -w job_id, --wait-job-id job_id
                        ID of job which must complete before starting this job
  -t walltime, --wall-time walltime
                        wallclock execution time in floating point hours,
                        defaults to 24.0

advanced arguments:
  --chain, --chain-jobs
                        each case depends on its predecessor
  --debug               debug mode
  --hostname host       force the hostname
  -r {overwrite,readwrite}, --restart {overwrite,readwrite}
                        optional restart mode
  --vera-installs-dir vera_installs_dir
                        path to vera_installs directory containing VERA
                        versions
```

3.1 VERARun Output

Upon completion of a VERA job, several output files may be created depending on the code options used. Some typical outputs include:

- VERAIn XML file. This file is written upon the successful completion of VERAIn
- VERA HDF output file. This is a binary file with results that can be visualized in VERAView, or post-processed with user utility codes.

- MPACT output file. This file is written upon the successful completion of MPACT (if applicable).
- MPACT log File. This file is written upon the successful completion of MPACT (if applicable).
- MPACT summary File. This file is written upon the successful completion of MPACT (if applicable).
- Standard output file. This file is a log of all output written to the standard output.
- Standard error file. this file is a log of all output written to the standard error file.

In case of errors, the user should examine the standard error file first. If there were any errors in the input processor (VERAIn), the errors will be written here. Any error messages from other VERA components will also be written to the standard error file.

If the standard error file does not list any errors, example the VERA component output files and log files for messages.